

[Home](#) > [4X](#) > [API](#) > Application[Edit on GitHub](#)

Application Object

The `app` object conventionally denotes the Express application. Create it by calling the top-level `express()` function exported by the Express module:

CommonJS

ESM

TypeScript

index.cjs

```
var express = require('express');
var app = express();

app.get('/', function (req, res) {
  res.send('hello world');
});

app.listen(3000);
```

The `app` object has methods for

- Routing HTTP requests; see for example, [app.METHOD](#) and [app.param](#).
- Configuring middleware; see [app.route](#).
- Rendering HTML views; see [app.render](#).
- Registering a template engine; see [app.engine](#).

It also has settings (properties) that affect how the application behaves; for more information, see [Application Settings](#).

 **Note**

The Express application object can be referred from the [request object](#) and the [response object](#) as `req.app`, and `res.app`, respectively.

Properties

`app.locals`

Type: `Object`

The `app.locals` object has properties that are local variables within the application, and will be available in templates rendered with [res.render](#).

Warning

The `locals` object is used by view engines to render a response. The object keys may be particularly sensitive and should not contain user-controlled input, as it may affect the operation of the view engine or provide a path to cross-site scripting. Consult the documentation for the used view engine for additional considerations.

```
console.dir(app.locals.title);  
// => 'My App'  
  
console.dir(app.locals.email);  
// => 'me@myapp.com'
```

Once set, the value of `app.locals` properties persist throughout the life of the application, in contrast with [res.locals](#) properties that are valid only for the lifetime of the request.

You can access local variables in templates rendered within the application. This is useful for providing helper functions to templates, as well as application-level data. Local variables are available in middleware via `req.app.locals` (see [req.app](#))

```
app.locals.title = 'My App';  
app.locals.strftime = require('strftime');  
app.locals.email = 'me@myapp.com';
```

`app.mountpath`

Type: `String | String[]`

The `app.mountpath` property contains one or more path patterns on which a sub-app was mounted.

Note

A sub-app is an instance of `express` that may be used for handling the request to a route.

CommonJS ESM TypeScript

index.cjs

```
var express = require('express');

var app = express(); // the main app
var admin = express(); // the sub app

admin.get('/', function (req, res) {
  console.log(admin.mountpath); // /admin
  res.send('Admin Homepage');
});

app.use('/admin', admin); // mount the sub app
```

It is similar to the [baseUrl](#) property of the `req` object, except `req.baseUrl` returns the matched URL path, instead of the matched patterns.

If a sub-app is mounted on multiple path patterns, `app.mountpath` returns the list of patterns it is mounted on, as shown in the following example.

JavaScript TypeScript

```
var admin = express();

admin.get('/', function (req, res) {
  console.dir(admin.mountpath); // [ '/adm*n', '/manager' ]
  res.send('Admin Homepage');
});

var secret = express();
```

```
secret.get('/', function (req, res) {
  console.log(secret.mountpath); // /secre*t
  res.send('Admin Secret');
});

admin.use('/secre*t', secret); // load the 'secret' router on '/secre*t', on
app.use(['/adm*n', '/manager'], admin); // load the 'admin' router on '/adm
```

Events

app.on('mount')

Arguments

callback

Type: Function

The listener called when the `mount` event fires. It receives the parent app as its only argument: `callback(parent)`.

The `mount` event is fired on a sub-app, when it is mounted on a parent app. The parent app is passed to the callback function.

i Note

Sub-apps will:

- Not inherit the value of settings that have a default value. You must set the value in the sub-app.
- Inherit the value of settings with no default value.

For details, see [Application settings](#).

JavaScript

TypeScript

```
var admin = express();

admin.on('mount', function (parent) {
  console.log('Admin Mounted');
  console.log(parent); // refers to the parent app
});
```

```
admin.get('/', function (req, res) {
  res.send('Admin Homepage');
});

app.use('/admin', admin);
```

Methods

app.all()

Arguments	
path	Type: String RegExp Array The path for which the middleware function is invoked; can be any of: a string representing a path, a path pattern, a regular expression pattern to match paths, or an array of combinations of any of the above. For examples, see Path examples.
callback	Type: Function Function[] Callback functions; can be: a middleware function, a series of middleware functions (separated by commas), an array of middleware functions, or a combination of all of the above. You can provide multiple callback functions that behave just like middleware, except that these callbacks can invoke <code>next('route')</code> to bypass the remaining route callback(s). You can use this mechanism to impose pre-conditions on a route, then pass control to subsequent routes if there is no reason to proceed with the current route. Since router and app implement the middleware interface, you can use them as you would any other middleware function. For examples, see Middleware callback function examples.

This method is like the standard `app.METHOD()` methods, except it matches all HTTP verbs.

Examples

On this page

JavaScript TypeScript

```
app.all('/secret', function (req, res, next) {
  console.log('Accessing the secret section ...');
  next(); // pass control to the next handler
});
```

The `app.all()` method is useful for mapping “global” logic for specific path prefixes or arbitrary matches. For example, if you put the following at the top of all other route definitions, it requires that all routes from that point on require authentication, and automatically load a user. Keep in mind that these callbacks do not have to act as end-points: `loadUser` can perform a task, then call `next()` to continue matching subsequent routes.

```
app.all('*', requireAuthentication, loadUser);
```

Or the equivalent:

```
app.all('*', requireAuthentication);
app.all('*', loadUser);
```

Another example is white-listed “global” functionality. The example is similar to the ones above, but it only restricts paths that start with “/api”:

```
app.all('/api/*', requireAuthentication);
```

app.delete()

Arguments

path

Type: `String | RegExp | Array`

The path for which the middleware function is invoked; can be any of: a string representing a path, a path pattern, a regular expression pattern to match paths, or an array of combinations of any of the above. For examples, see [Path examples](#).

callback

Type: `Function | Function[]`

Callback functions; can be: a middleware function, a series of middleware functions (separated by commas), an array of middleware functions, or a combination of all of the above. You can

provide multiple callback functions that behave just like middleware, except that these callbacks can invoke `next('route')` to bypass the remaining route callback(s). You can use this mechanism to impose pre-conditions on a route, then pass control to subsequent routes if there is no reason to proceed with the current route. Since [router](#) and [app](#) implement the middleware interface, you can use them as you would any other middleware function. For examples, see [Middleware callback function examples](#).

Routes HTTP DELETE requests to the specified path with the specified callback functions. For more information, see the [routing guide](#).

Example

JavaScript TypeScript

```
app.delete('/', function (req, res) {  
  res.send('DELETE request to homepage');  
});
```

app.del()

Arguments	
path	Type: String RegExp Array The path for which the middleware function is invoked; same as app.delete().
callback	Type: Function Function[] Callback functions; same as app.delete().
Deprecated in: v4.2.0	

 Caution

`app.del()` is the alias of [app.delete\(\)](#), originally introduced because `delete` was a reserved word in older JavaScript engines. It has emitted a deprecation warning since Express v4.2.0. Use `app.delete()` instead.
You can update your code automatically with the following codemod:

npm yarn pnpm bun

npx codemod@latest @expressjs/route-del-to-delete

app.disable()

Arguments	
name	Type: String The name of the Boolean setting to disable; one of the properties from the app settings table.
↩ Returns: Application	

Sets the Boolean setting `name` to `false`, where `name` is one of the properties from the [app settings table](#). Calling `app.set('foo', false)` for a Boolean property is the same as calling `app.disable('foo')`.

For example:

```
app.disable('trust proxy');
app.get('trust proxy');
// => false
```

app.disabled()

Arguments	
name	Type: String The name of the Boolean setting to test; one of the properties from the app settings table.
↩ Returns: Boolean	

Returns `true` if the Boolean setting `name` is disabled (`false`), where `name` is one of the properties from the [app settings table](#).

```
app.disabled('trust proxy');  
// => true  
  
app.enable('trust proxy');  
app.disabled('trust proxy');  
// => false
```

app.enable()

Arguments

name**Type:** String

The name of the Boolean setting to enable; one of the properties from the [app settings table](#).

↩ Returns: Application

Sets the Boolean setting `name` to `true`, where `name` is one of the properties from the [app settings table](#). Calling `app.set('foo', true)` for a Boolean property is the same as calling `app.enable('foo')`.

```
app.enable('trust proxy');  
app.get('trust proxy');  
// => true
```

app.enabled()

Arguments

name**Type:** String

The name of the setting to test; one of the properties from the [app settings table](#).

↩ Returns: Boolean

Returns `true` if the setting `name` is enabled (`true`), where `name` is one of the properties from the [app settings table](#).

```
app.enabled('trust proxy');
```

```
// => false

app.enable('trust proxy');
app.enabled('trust proxy');
// => true
```

app.engine()

Arguments

ext**Type:** `String`

The file extension the template engine is registered for (for example, pug or html).

callback**Type:** `Function`

The template engine function used to render files with the given extension.

↳ Returns: `Application`

Registers the given template engine `callback` as `ext`.

By default, Express will `require()` the engine based on the file extension. For example, if you try to render a “foo.pug” file, Express invokes the following internally, and caches the `require()` on subsequent calls to increase performance.

```
app.engine('pug', require('pug').__express);
```

Use this method for engines that do not provide `.__express` out of the box, or if you wish to “map” a different extension to the template engine.

For example, to map the EJS template engine to “.html” files:

```
app.engine('html', require('ejs').renderFile);
```

In this case, EJS provides a `.renderFile()` method with the same signature that Express expects: `(path, options, callback)`, though note that it aliases this method as `ejs.__express` internally so if you’re using “.ejs” extensions you don’t need to do anything.

Some template engines do not follow this convention. The [consolidate.js](#) library maps Node template engines to follow this convention, so they work seamlessly with Express.

CommonJS ESM

index.cjs

```
var engines = require('consolidate');
app.engine('haml', engines.haml);
app.engine('html', engines.hogan);
```

app.get(name)

Arguments	
name	Type: String The name of the setting to read; one of the strings in the app settings table.
Returns: any	

Returns the value of `name` app setting, where `name` is one of the strings in the [app settings table](#). For example:

```
app.get('title');
// => undefined

app.set('title', 'My Site');
app.get('title');
// => "My Site"
```

app.get()

Arguments	
path	Type: String RegExp Array The path for which the middleware function is invoked; can be any of: a string representing a path, a path pattern, a regular expression pattern to match paths, or an array of combinations of any of the above. For examples, see Path examples.

callback

Type:

Function | Function[]

Callback functions; can be: a middleware function, a series of middleware functions (separated by commas), an array of middleware functions, or a combination of all of the above. You can provide multiple callback functions that behave just like middleware, except that these callbacks can invoke `next('route')` to bypass the remaining route callback(s). You can use this mechanism to impose pre-conditions on a route, then pass control to subsequent routes if there is no reason to proceed with the current route. Since [router](#) and [app](#) implement the middleware interface, you can use them as you would any other middleware function. For examples, see [Middleware callback function examples](#).

Routes HTTP GET requests to the specified path with the specified callback functions.

For more information, see the [routing guide](#).

Example

JavaScript

TypeScript

```
app.get('/', function (req, res) {  
  res.send('GET request to homepage');  
});
```

app.listen()

Arguments	
port	Type: Number undefined The TCP port to listen on. If omitted or <code>0</code>, the OS assigns an arbitrary unused port.
host	Type: String undefined The host on which to accept connections.
backlog	Type: Number undefined The maximum length of the queue of pending connections.

path	Type: <code>String undefined</code>
The path of a UNIX socket to listen on, as an alternative to host and port.	
callback	Type: <code>Function undefined</code>
A function to call once the server is listening.	
↵ Returns:	<code>http.Server</code>

Binds and listens for connections on the specified host and port, or starts a UNIX socket and listens for connections on the given path. This method is identical to Node's [http.Server.listen\(\)](#).

If port is omitted or is 0, the operating system will assign an arbitrary unused port, which is useful for cases like automated tasks (tests, etc.).

CommonJS	ESM	TypeScript
index.cjs <pre>var express = require('express'); var app = express(); app.listen(3000); // bind and listen on a TCP port app.listen('/tmp/sock'); // or listen on a UNIX socket</pre>		

The `app` returned by `express()` is in fact a JavaScript Function, designed to be passed to Node's HTTP servers as a callback to handle requests. This makes it easy to provide both HTTP and HTTPS versions of your app with the same code base, as the `app` does not inherit from these (it is simply a callback):

CommonJS	ESM	TypeScript
index.cjs <pre>var express = require('express'); var https = require('https'); var http = require('http'); var app = express(); http.createServer(app).listen(80); https.createServer(options, app).listen(443);</pre>		

The `app.listen()` method returns an [http.Server](#) object and (for HTTP) is a convenience method for the following:

```
app.listen = function () {  
  var server = http.createServer(this);  
  return server.listen.apply(server, arguments);  
};
```

Note

All the forms of Node's [http.Server.listen\(\)](#) method are in fact actually supported.

app.METHOD()

Arguments

path

Type: `String | RegExp | Array`

The path for which the middleware function is invoked; can be any of: a string representing a path, a path pattern, a regular expression pattern to match paths, or an array of combinations of any of the above. For examples, see [Path examples](#).

callback

Type: `Function | Function[]`

Callback functions; can be: a middleware function, a series of middleware functions (separated by commas), an array of middleware functions, or a combination of all of the above. You can provide multiple callback functions that behave just like middleware, except that these callbacks can invoke `next('route')` to bypass the remaining route callback(s). You can use this mechanism to impose pre-conditions on a route, then pass control to subsequent routes if there is no reason to proceed with the current route. Since [router](#) and [app](#) implement the middleware interface, you can use them as you would any other middleware function. For examples, see [Middleware callback function examples](#).

Routes an HTTP request, where METHOD is the HTTP method of the request, such as GET, PUT, POST, and so on, in lowercase. Thus, the actual methods are `app.get()`, `app.post()`, `app.put()`, and so on. See [Routing methods](#) below for the complete list.

Routing methods

Express supports the following routing methods corresponding to the HTTP methods of the same names:

- checkout
- copy
- [delete](#)
- [get](#)
- head
- lock
- merge
- mkactivity
- mkcol
- move
- m-search
- notify
- options
- patch
- [post](#)
- purge
- [put](#)
- [query](#)
- report
- search
- subscribe
- trace
- unlock
- unsubscribe

The API documentation has explicit entries only for the most popular HTTP methods `app.get()` , `app.post()` , `app.put()` , `app.delete()` , and `app.query()` . However, the other methods listed above work in exactly the same way.

To route methods that translate to invalid JavaScript variable names, use the bracket notation. For example, `app['m-search']('/', function ...` .

JavaScript	TypeScript
<pre>app.get('/', function (req, res) { res.send('GET request to homepage'); }); app.post('/', function (req, res) { res.send('POST request to homepage'); }); // methods that are invalid JavaScript variable names use bracket notation +app['m-search']('/', function (req, res) { res.send('M-SEARCH request to homepage'); });</pre>	



The `app.get()` function is automatically called for the HTTP `HEAD` method in addition to the `GET` method if `app.head()` was not called for the path before `app.get()`.

The method, `app.all()`, is not derived from any HTTP method and loads middleware at the specified path for *all* HTTP request methods. For more information, see [app.all](#).

For more information on routing, see the [routing guide](#).

app.query()

Arguments	
path	Type: String RegExp Array The path for which the middleware function is invoked; can be any of: a string representing a path, a path pattern, a regular expression pattern to match paths, or an array of combinations of any of the above. For examples, see Path examples.
callback	Type: Function Function[] Callback functions; can be: a middleware function, a series of middleware functions (separated by commas), an array of middleware functions, or a combination of all of the above. You can provide multiple callback functions that behave just like middleware, except that these callbacks can invoke <code>next('route')</code> to bypass the remaining route callback(s). You can use this mechanism to impose pre-conditions on a route, then pass control to subsequent routes if there is no reason to proceed with the current route. Since router and app implement the middleware interface, you can use them as you would any other middleware function. For examples, see Middleware callback function examples.
Requires runtime: Node.js ≥ 20.19.3 <21 ≥ 22.2.0	

Routes HTTP `QUERY` requests to the specified path with the specified callback functions. For more information, see the [routing guide](#).

 Caution

The HTTP `QUERY` method was standardized on June 15, 2026 as [RFC 10008](#). Client and intermediary support is still limited. Verify that your target environments support the `QUERY` verb before relying on it in production.

The HTTP QUERY method ([RFC 10008](#)) is designed for queries whose input is too large or too sensitive to encode in the URL. Unlike GET , a QUERY request carries a body containing the query input. Unlike POST , it is explicitly **safe and idempotent**, intermediaries and caches can retry it automatically, and the response is cacheable (with the request body included in the cache key).

Use `app.query()` when query parameters would exceed URL length limits, contain sensitive data that should not appear in server logs or browser history, or are too complex to express cleanly as a URL query string, such as deeply nested filters or structured criteria.

Because the query input travels in the request body, `req.body` is undefined until you mount a body-parsing middleware such as [express.json\(\)](#) or [express.urlencoded\(\)](#) that matches the request's Content-Type .

Example

JavaScript TypeScript

index.js

```
app.use(express.json()); // populate req.body for application/json payloads

app.query('/search', (req, res) => {
  // complex filter criteria arrive in req.body, not the URL
  const results = db.search(req.body);
  res.json(results);
});
```

app.param()

Arguments	
name	Type: String String[] undefined The name of the route parameter, or an array of names.
callback	Type: Function The trigger called as <code>callback(req, res, next, value, name)</code> : the request object, the response object, the next middleware, the value of the parameter, and the name of the parameter, in that order.

Add callback triggers to [route parameters](#), where `name` is the name of the parameter or an array of them, and `callback` is the callback function. The parameters of the callback function are the request object, the response object, the next middleware, the value of the parameter and the name of the parameter, in that order.

If `name` is an array, the `callback` trigger is registered for each parameter declared in it, in the order in which they are declared. Furthermore, for each declared parameter except the last one, a call to `next` inside the callback will call the callback for the next declared parameter. For the last parameter, a call to `next` will call the next middleware in place for the route currently being processed, just like it would if `name` were just a string.

For example, when `:user` is present in a route path, you may map user loading logic to automatically provide `req.user` to the route, or perform validations on the parameter input.

```
app.param('user', function (req, res, next, id) {  
  // try to get the user details from the User model and attach it to the r  
  User.find(id, function (err, user) {  
    if (err) {  
      next(err);  
    } else if (user) {  
      req.user = user;  
      next();  
    } else {  
      next(new Error('failed to load user'));  
    }  
  });  
});
```

Param callback functions are local to the router on which they are defined. They are not inherited by mounted apps or routers, nor are they triggered for route parameters inherited from parent routers. Hence, param callbacks defined on `app` will be triggered only by route parameters defined on `app` routes.

All param callbacks will be called before any handler of any route in which the param occurs, and they will each be called only once in a request-response cycle, even if the parameter is matched in multiple routes, as shown in the following examples.

JavaScript	TypeScript
<pre>app.param('id', function (req, res, next, id) { console.log('CALLED ONLY ONCE'); next(); });</pre>	

```
});

app.get('/user/:id', function (req, res, next) {
  console.log('although this matches');
  next();
});

app.get('/user/:id', function (req, res) {
  console.log('and this matches too');
  res.end();
});
```

On GET /user/42 , the following is printed:

```
CALLED ONLY ONCE
although this matches
and this matches too
```

JavaScript TypeScript

```
app.param(['id', 'page'], function (req, res, next, value) {
  console.log('CALLED ONLY ONCE with', value);
  next();
});

app.get('/user/:id/:page', function (req, res, next) {
  console.log('although this matches');
  next();
});

app.get('/user/:id/:page', function (req, res) {
  console.log('and this matches too');
  res.end();
});
```

On GET /user/42/3 , the following is printed:

```
CALLED ONLY ONCE with 42
CALLED ONLY ONCE with 3
although this matches
and this matches too
```

 Caution

The following section describes `app.param(callback)`, which is deprecated as of v4.11.0.

The behavior of the `app.param(name, callback)` method can be altered entirely by passing only a function to `app.param()`. This function is a custom implementation of how `app.param(name, callback)` should behave - it accepts two parameters and must return a middleware.

The first parameter of this function is the name of the URL parameter that should be captured, the second parameter can be any JavaScript object which might be used for returning the middleware implementation.

The middleware returned by the function decides the behavior of what happens when a URL parameter is captured.

In this example, the `app.param(name, callback)` signature is modified to `app.param(name, accessId)`. Instead of accepting a name and a callback, `app.param()` will now accept a name and a number.

CommonJS

ESM

index.cjs

```
var express = require('express');
var app = express();

// customizing the behavior of app.param()
app.param(function (param, option) {
  return function (req, res, next, val) {
    if (val === option) {
      next();
    } else {
      next('route');
    }
  };
});

// using the customized app.param()
app.param('id', 1337);

// route to trigger the capture
app.get('/user/:id', function (req, res) {
```

```
    res.send('OK');
  });

app.listen(3000, function () {
  console.log('Ready');
});
```

In this example, the `app.param(name, callback)` signature remains the same, but instead of a middleware callback, a custom data type checking function has been defined to validate the data type of the user id.

```
app.param(function (param, validator) {
  return function (req, res, next, val) {
    if (validator(val)) {
      next();
    } else {
      next('route');
    }
  };
});

app.param('id', function (candidate) {
  return !isNaN(parseFloat(candidate)) && isFinite(candidate);
});
```

Note

The `.` character can't be used to capture a character in your capturing regexp. For example you can't use `/user-.+/'` to capture `'users-gami'`, use `[\s\S]` or `[\w\W]` instead (as in `/user-[\s\S]+/'`).

Examples:

JavaScript	TypeScript
<pre>// captures '1-a_6' but not '543-azser-sder' router.get('/[0-9]+-[[\w]]*', function (req, res, next) { next(); }); // captures '1-a_6' and '543-az(ser"-sder' but not '5-a s' router.get('/[0-9]+-[[\S]]*', function (req, res, next) { next(); });</pre>	

```
});

// captures all (equivalent to '.*')
router.get('[\s\S]*', function (req, res, next) {
  next();
});
```

app.path()

↵ Returns: `String`

Returns the canonical path of the app, a string.

JavaScript TypeScript

index.js

```
var app = express();
var blog = express();
var blogAdmin = express();

app.use('/blog', blog);
blog.use('/admin', blogAdmin);

console.dir(app.path()); // ''
console.dir(blog.path()); // '/blog'
console.dir(blogAdmin.path()); // '/blog/admin'
```

The behavior of this method can become very complicated in complex cases of mounted apps: it is usually better to use [req.baseUrl](#) to get the canonical path of the app.

app.post()

Arguments

path

Type: `String | RegExp | Array`

The path for which the middleware function is invoked; can be any of: a string representing a path, a path pattern, a regular expression

pattern to match paths, or an array of combinations of any of the above. For examples, see Path examples .	
callback	Type: <code>Function Function[]</code> Callback functions; can be: a middleware function, a series of middleware functions (separated by commas), an array of middleware functions, or a combination of all of the above. You can provide multiple callback functions that behave just like middleware, except that these callbacks can invoke <code>next('route')</code> to bypass the remaining route callback(s). You can use this mechanism to impose pre-conditions on a route, then pass control to subsequent routes if there is no reason to proceed with the current route. Since router and app implement the middleware interface, you can use them as you would any other middleware function. For examples, see Middleware callback function examples.

Routes HTTP POST requests to the specified path with the specified callback functions. For more information, see the [routing guide](#).

Example

JavaScript TypeScript

```
app.post('/', function (req, res) {  
  res.send('POST request to homepage');  
});
```

app.put()

Arguments	
path	Type: <code>String RegExp Array</code> The path for which the middleware function is invoked; can be any of: a string representing a path, a path pattern, a regular expression pattern to match paths, or an array of combinations of any of the above. For examples, see Path examples.
callback	Type: <code>Function Function[]</code> Callback functions; can be: a middleware function, a series of middleware functions (separated by commas), an array of

middleware functions, or a combination of all of the above. You can provide multiple callback functions that behave just like middleware, except that these callbacks can invoke `next('route')` to bypass the remaining route callback(s). You can use this mechanism to impose pre-conditions on a route, then pass control to subsequent routes if there is no reason to proceed with the current route. Since [router](#) and [app](#) implement the middleware interface, you can use them as you would any other middleware function. For examples, see [Middleware callback function examples](#).

Routes HTTP PUT requests to the specified path with the specified callback functions.

Example

JavaScript TypeScript

```
app.put('/', function (req, res) {  
  res.send('PUT request to homepage');  
});
```

app.render()

Arguments	
view	Type: String The name of the view to render.
locals	Type: Object undefined An object whose properties define local variables for the view.
callback	Type: Function The callback invoked as <code>callback(err, html)</code> with the rendered HTML.

Returns the rendered HTML of a view via the `callback` function. It accepts an optional parameter that is an object containing local variables for the view. It is like [res.render\(\)](#), except it cannot send the rendered view to the client on its own.



Note

Think of `app.render()` as a utility function for generating rendered view strings. Internally `res.render()` uses `app.render()` to render views.

 Warning

The `view` argument performs file system operations like reading a file from disk and evaluating Node.js modules, and as so for security reasons should not contain input from the end-user.

 Warning

The `locals` object is used by view engines to render a response. The object keys may be particularly sensitive and should not contain user-controlled input, as it may affect the operation of the view engine or provide a path to cross-site scripting. Consult the documentation for the used view engine for additional considerations.

 Caution

The local variable `cache` is reserved for enabling view cache. Set it to `true`, if you want to cache view during development; view caching is enabled in production by default.

```
app.render('email', function (err, html) {
  // ...
});

app.render('email', { name: 'Tobi' }, function (err, html) {
  // ...
});
```

app.route()

Arguments	
path	Type: String The path of the route to create.
↵ Returns:	Route

Returns an instance of a single route, which you can then use to handle HTTP verbs with optional middleware. Use `app.route()` to avoid duplicate route names (and thus

typo errors).

JavaScript TypeScript

```
var app = express();

app
  .route('/events')
  .all(function (req, res, next) {
    // runs for all HTTP verbs first
    // think of it as route specific middleware!
  })
  .get(function (req, res, next) {
    res.json({});
  })
  .post(function (req, res, next) {
    // maybe add a new event...
  });
```

app.set()

Arguments	
name	Type: String The name of the setting to assign. Certain names configure the server's behavior; see the app settings table .
value	Type: any The value to assign to the setting.
↩ Returns: Application	

Assigns setting `name` to `value`. You may store any value that you want, but certain names can be used to configure the behavior of the server. These special names are listed in the [app settings table](#).

Calling `app.set('foo', true)` for a Boolean property is the same as calling `app.enable('foo')`. Similarly, calling `app.set('foo', false)` for a Boolean property is the same as calling `app.disable('foo')`.

Retrieve the value of a setting with [app.get\(\)](#).

```
app.set('title', 'My Site');
app.get('title'); // "My Site"
```

Application Settings

The following table lists application settings.

i

Note

Sub-apps will:

- Not inherit the value of settings that have a default value. You must set the value in the sub-app.
- Inherit the value of settings with no default value; these are explicitly noted in the table below.

Exceptions: Sub-apps will inherit the value of `trust proxy` even though it has a default value (for backward-compatibility); Sub-apps will not inherit the value of `view cache` in production (when `NODE_ENV` is “production”).

Settings		
case sensitive routing	Type: Boolean Default: N/A (undefined) Enable case sensitivity. When enabled, “/Foo” and “/foo” are different routes. When disabled, “/Foo” and “/foo” are treated the same. i Note Sub-apps will inherit the value of this setting.	
env	Type: String Default: process.env.NODE_ENV (NODE_ENV environment variable) or "development" if NODE_ENV is not set. Environment mode. Be sure to set to “production” in a production environment; see Production best practices: performance and reliability.	
etag	Type: Boolean String Function Default: weak	

	Set the ETag response header. For possible values, see the <code>etag</code> options table below. More about the HTTP ETag header .	
<code>jsonp callback name</code>	Type: <code>String</code> Default: <code>"callback"</code>	Specifies the default JSONP callback name.
<code>json escape</code>	Type: <code>Boolean</code> Default: <code>N/A (undefined)</code>	Enable escaping JSON responses from the <code>res.json</code> , <code>res.jsonp</code> , and <code>res.send</code> APIs. This will escape the characters <code><</code> , <code>></code> , and <code>&</code> as Unicode escape sequences in JSON. The purpose of this is to assist with mitigating certain types of persistent XSS attacks when clients sniff responses for HTML. Note Sub-apps will inherit the value of this setting.
<code>json replacer</code>	Type: <code>Function Array</code> Default: <code>N/A (undefined)</code>	The ‘replacer’ argument used by JSON.stringify . Note Sub-apps will inherit the value of this setting.
<code>json spaces</code>	Type: <code>Number String</code> Default: <code>N/A (undefined)</code>	The ‘space’ argument used by JSON.stringify . This is typically set to the number of spaces to use to indent prettified JSON. Note Sub-apps will inherit the value of this setting.
<code>query parser</code>	Type: <code>Boolean String Function</code> Default: <code>"extended"</code>	Disable query parsing by setting the value to <code>false</code> , or set the query parser to use either “simple” or “extended” or a custom query string parsing function. The simple query parser is based on Node’s native query parser, querystring . The extended query parser is based on qs . A custom query string parsing function will receive

	the complete query string, and must return an object of query keys and their values.	
strict routing	Type: <code>Boolean</code> Default: <code>N/A (undefined)</code> Enable strict routing. When enabled, the router treats <code>"/foo"</code> and <code>"/foo/"</code> as different. Otherwise, the router treats <code>"/foo"</code> and <code>"/foo/"</code> as the same. <i>i</i> Note Sub-apps will inherit the value of this setting.	
subdomain offset	Type: <code>Number</code> Default: <code>2</code> The number of dot-separated parts of the host to remove to access subdomain.	
trust proxy	Type: <code>Boolean String Number Function Array</code> Default: <code>false (disabled)</code> Indicates the app is behind a front-facing proxy, and to use the <code>X-Forwarded-*</code> headers to determine the connection and the IP address of the client. <i>i</i> Note X-Forwarded-* headers are easily spoofed and the detected IP addresses are unreliable. When enabled, Express attempts to determine the IP address of the client connected through the front-facing proxy, or series of proxies. The <code>req.ips</code> property, then contains an array of IP addresses the client is connected through. To enable it, use the values described in the trust proxy options table below. The <code>trust proxy</code> setting is implemented using the proxy-addr package. For more information, see its documentation. <i>i</i> Note Sub-apps <i>will</i> inherit the value of this setting, even though it has a default value.	

views	Type: <code>String Array</code>	Default: <code>process.cwd() + '/views'</code>
A directory or an array of directories for the application's views. If an array, the views are looked up in the order they occur in the array.		
view cache	Type: <code>Boolean</code>	Default: <code>true</code> in production, otherwise <code>undefined</code> .
Enables view template compilation caching.		
Note Sub-apps will not inherit the value of this setting in production (when <code>NODE_ENV</code> is "production").		
view engine	Type: <code>String</code>	Default: <code>N/A (undefined)</code>
The default engine extension to use when omitted.		
Note Sub-apps will inherit the value of this setting.		
x-powered-by	Type: <code>Boolean</code>	Default: <code>true</code>
Enables the "X-Powered-By: Express" HTTP header.		

Options for trust proxy setting

Read [Express behind proxies](#) for more information.

Boolean: If `true`, the client's IP address is understood as the left-most entry in the `X-Forwarded-*` header. If `false`, the app is understood as directly facing the Internet and the client's IP address is derived from `req.connection.remoteAddress`. This is the default setting.

String / String containing comma-separated values / Array of strings: An IP address, subnet, or an array of IP addresses, and subnets to trust. Pre-configured subnet names are:

- `loopback` - `127.0.0.1/8`, `::1/128`
- `linklocal` - `169.254.0.0/16`, `fe80::/10`
- `uniquelocal` - `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `fc00::/7`

Set IP addresses in any of the following ways:

Specify a single subnet:

```
app.set('trust proxy', 'loopback');
```

Specify a subnet and an address:

```
app.set('trust proxy', 'loopback, 123.123.123.123');
```

Specify multiple subnets as CSV:

```
app.set('trust proxy', 'loopback, linklocal, uniquelocal');
```

Specify multiple subnets as an array:

```
app.set('trust proxy', ['loopback', 'linklocal', 'uniquelocal']);
```

When specified, the IP addresses or the subnets are excluded from the address determination process, and the untrusted IP address nearest to the application server is determined as the client's IP address.

Number: Trust the *n*th hop from the front-facing proxy server as the client.

Function: Custom trust implementation. Use this only if you know what you are doing.

```
app.set('trust proxy', function (ip) {  
  if (ip === '127.0.0.1' || ip === '123.123.123.123')  
    return true; // trusted IPs  
  else return false;  
});
```

Options for etag setting

Note

These settings apply only to dynamic files, not static files. The `express.static` middleware ignores these settings.

The ETag functionality is implemented using the [etag](#) package. For more information, see its documentation.

Values	
Boolean	true enables weak ETag. This is the default setting. false disables ETag altogether.
String	If “strong”, enables strong ETag. If “weak”, enables weak ETag.
Function	Custom ETag function implementation. Use this only if you know what you are doing.

```
app.set('etag', function (body, encoding) {  
  return generateHash(body, encoding); // consider the function is defined  
});
```

app.use()

Arguments	
path	Type: String RegExp Array undefined The path for which the middleware function is invoked; can be any of: a string representing a path, a path pattern, a regular expression pattern to match paths, or an array of combinations of any of the above. For examples, see Path examples.
callback	Type: Function Function[] Callback functions; can be: a middleware function, a series of middleware functions (separated by commas), an array of middleware functions, or a combination of all of the above. You can provide multiple callback functions that behave just like middleware, except that these callbacks can invoke <code>next('route')</code> to bypass the remaining route callback(s). You can use this mechanism to impose pre-conditions on a route, then pass control to subsequent routes if there is no reason to proceed with the current route. Since router and app implement the middleware interface, you can use them as you would any other middleware function. For examples, see Middleware callback function examples.

Mounts the specified [middleware](#) function or functions at the specified path: the middleware function is executed when the base of the requested path matches `path`.

Description

A route will match any path that follows its path immediately with a `/`. For example:

`app.use('/apple', ...)` will match `"/apple"`, `"/apple/images"`, `"/apple/images/news"`, and so on.

Since `path` defaults to `"/"`, middleware mounted without a path will be executed for every request to the app. For example, this middleware function will be executed for *every* request to the app:

JavaScript	TypeScript
<pre>app.use(function (req, res, next) { console.log('Time: %d', Date.now()); next(); });</pre>	

Note

Sub-apps will:

- Not inherit the value of settings that have a default value. You must set the value in the sub-app.
- Inherit the value of settings with no default value.

For details, see [Application settings](#).

Middleware functions are executed sequentially, therefore the order of middleware inclusion is important.

JavaScript	TypeScript
<pre>// this middleware will not allow the request to go beyond it app.use(function (req, res, next) { res.send('Hello World'); }); // requests will never reach this route app.get('/', function (req, res) { res.send('Welcome'); });</pre>	

```
});
```

Error-handling middleware

Error-handling middleware always takes *four* arguments. You must provide four arguments to identify it as an error-handling middleware function. Even if you don't need to use the `next` object, you must specify it to maintain the signature. Otherwise, the `next` object will be interpreted as regular middleware and will fail to handle errors. For details about error-handling middleware, see: [Error handling](#).

Define error-handling middleware functions in the same way as other middleware functions, except with four arguments instead of three, specifically with the signature `(err, req, res, next)`:

JavaScript	TypeScript
<pre>app.use(function (err, req, res, next) { console.error(err.stack); res.status(500).send('Something broke!'); });</pre>	

Path examples

The following table provides some simple examples of valid `path` values for mounting middleware.

Path: Matches the exact path `/abcd` and any sub-paths starting with `/abcd/` (for example, `/abcd/foo`):

JavaScript	TypeScript
<pre>app.use('/abcd', function (req, res, next) { next(); });</pre>	

Path Pattern: This will match paths starting with `/abcd` and `/abd`:

JavaScript	TypeScript
<pre>app.use('/abc?d', function (req, res, next) { next(); });</pre>	

```
});
```

This will match paths starting with `/abcd` , `/abbcd` , `/abbbbbcd` , and so on:

JavaScript TypeScript

```
app.use('/ab+cd', function (req, res, next) {  
  next();  
});
```



This will match paths starting with `/abcd` , `/abxcd` , `/abF00cd` , `/abbArcd` , and so on:

JavaScript TypeScript

```
app.use('/ab*cd', function (req, res, next) {  
  next();  
});
```



This will match paths starting with `/ad` and `/abcd` :

JavaScript TypeScript

```
app.use('/a(bc)?d', function (req, res, next) {  
  next();  
});
```



Regular Expression: This will match paths starting with `/abc` and `/xyz` :

JavaScript TypeScript

```
app.use(/\/abc|\/xyz/, function (req, res, next) {  
  next();  
});
```



Array: This will match paths starting with `/abcd` , `/xyza` , `/lmn` , and `/pqr` :

JavaScript TypeScript

```
app.use(['/abcd', '/xyza', /\lmn|\pqr/], function (req, res, next) {  
  next();  
});
```

Middleware callback function examples

The following table provides some simple examples of middleware functions that can be used as the `callback` argument to `app.use()`, `app.METHOD()`, and `app.all()`.

Single Middleware: You can define and mount a middleware function locally.

JavaScript	TypeScript
<pre>app.use(function (req, res, next) { next(); });</pre>	

A router is valid middleware.

JavaScript	TypeScript
<pre>var router = express.Router(); router.get('/', function (req, res, next) { next(); }); app.use(router);</pre>	

An Express app is valid middleware.

JavaScript	TypeScript
<pre>var subApp = express(); subApp.get('/', function (req, res, next) { next(); }); app.use(subApp);</pre>	

Series of Middleware: You can specify more than one middleware function at the same mount path.

JavaScript	TypeScript
<pre>var r1 = express.Router(); r1.get('/', function (req, res, next) { next(); }); var r2 = express.Router(); r2.get('/', function (req, res, next) { next(); }); app.use(r1, r2);</pre>	☐

Array: Use an array to group middleware logically.

JavaScript	TypeScript
<pre>var r1 = express.Router(); r1.get('/', function (req, res, next) { next(); }); var r2 = express.Router(); r2.get('/', function (req, res, next) { next(); }); app.use([r1, r2]);</pre>	☐

Combination: You can combine all the above ways of mounting middleware.

JavaScript	TypeScript
<pre>function mw1(req, res, next) { next(); } function mw2(req, res, next) { next(); } var r1 = express.Router();</pre>	☐

```
r1.get('/', function (req, res, next) {
  next();
});

var r2 = express.Router();
r2.get('/', function (req, res, next) {
  next();
});

var subApp = express();
subApp.get('/', function (req, res, next) {
  next();
});

app.use(mw1, [mw2, r1, r2], subApp);
```

Following are some examples of using the [express.static](#) middleware in an Express app.

Serve static content for the app from the “public” directory in the application directory:

```
// GET /style.css etc
app.use(express.static(path.join(__dirname, 'public')));
```

Mount the middleware at “/static” to serve static content only when their request path is prefixed with “/static”:

```
// GET /static/style.css etc.
app.use('/static', express.static(path.join(__dirname, 'public')));
```

Disable logging for static content requests by loading the logger middleware after the static middleware:

```
app.use(express.static(path.join(__dirname, 'public')));
app.use(logger());
```

Serve static files from multiple directories, but give precedence to “./public” over the others:

```
app.use(express.static(path.join(__dirname, 'public')));
app.use(express.static(path.join(__dirname, 'files')));
app.use(express.static(path.join(__dirname, 'uploads')));
```

← Previous
express()

Next →
Request

Copyright [OpenJS Foundation](#) and Express.js contributors. All rights reserved. The [OpenJS Foundation](#) has registered trademarks and uses trademarks. For a list of trademarks of the [OpenJS Foundation](#), please see our [Trademark Policy](#) and [Trademark List](#). Trademarks and logos not indicated on the [list of OpenJS Foundation trademarks](#) are trademarks™ or registered® trademarks of their respective holders. Use of them does not imply any affiliation with or endorsement by them.

[The OpenJS Foundation](#) [AI Coding Assistants Policy](#) [Bylaws](#) [Code of Conduct](#) [Cookie Policy](#) [Privacy Policy](#) [Security Policy](#) [Terms of Use](#) [Trademark List](#) [Trademark Policy](#)



Kawaii?



